Emergency Training Simulator POC

Markdown documentation PDF copy

Emergency Training Simulator POC: one-page handout

Source: docs/presentations/project-overview-one-page-handout.md

Markdown stays editable. This PDF is the reading/share copy.

Boundary: this project is still a POC, not operational guidance.

Emergency Training Simulator POC: one-page handout

What this is

This repo is a backend-first proof of concept for an AI-driven adaptive emergency-response training simulator.

The first audience is Dr. Cameron, so the project is framed as an on-campus job/research-style progress update: what I built, what it proves, what it does not prove yet, and what the next careful step is.

The full project vision is larger than this repo. It points toward AI-driven scenario adaptation, immersive environments, multimodal sensing, validation, and scalable deployment. This repo proves the first backend/replay/review foundation underneath that larger direction.

It proves a small but important loop:

```
trainee action -> backend -> scenario state -> validation -> client update -> log -> replay/review
```

What it proves today

- A FastAPI backend can receive ordered trainee actions over WebSocket.
- A deterministic scenario engine can update state from a YAML scenario.
- POC validation rules can return ok, warning, or error.
- Basic POC security guardrails reject unsafe client input.
- JSONL session logs can be written locally.
- Replay and review scripts can explain what happened after the run.
- A future Unreal client can follow a simple message contract.

Current scenario

The current scenario is a tiny smoke-room POC.

It includes:

- a wooden_chair_poc object that may block the path
- an entry_door_poc object that starts closed
- actions like move_obstacle and open_door

This is a software scenario only. It is not medical, firefighter, EMT, or operational guidance.

Security baseline

The client is not trusted as the source of truth.

A client can request a trainee action. The backend decides the state change, validation result, final outcome, log entry, replay result, and review summary.

Current guardrails reject strange IDs, path-like scenario names, unexpected message fields, unsafe action names, oversized payloads, wrong sequence numbers, and duplicate event IDs.

This is not production security yet. There is no auth, TLS, cloud hardening, or real data handling policy.

Current demo

Run:

```
python scripts/run_local_demo.py
```

The demo runs:

- a happy path
- a warning/error path
- replay verification
- review summaries
- wooden-chair physics preview metadata

Important boundary

This repo does not currently include:

- real AI or API calls
- clinical validation
- official emergency protocol authority
- generated 3D assets
- real physics
- a full Unreal scene
- sensors
- cloud deployment

Next direction

The next work should keep the backend as the source of truth while making the future client contract clearer.

Near-term focus:

1. strengthen the display contract
2. keep tightening the security boundary
3. keep replay and review central
4. add richer environment object metadata carefully
5. bring in Unreal only when the text/display contract is stable
6. use Blender for manual review before any generated asset is trusted

Ideal end result

The long-term goal is an adaptive training simulator where environment cues matter, instructors can review sessions clearly, and future AI/3D/Unreal pieces support the training experience without hiding the system logic.

Suggested 10-minute presentation

- 5 minutes: explain the loop, scope, validation boundary, and security boundary
- 5 minutes: run the local demo and point at replay/review

Main takeaway:

The backend can keep track of what happened, explain it afterward, and leave room for Unreal, UX, assets, and p